INT161 · Basic Backend Development

Error Handling

ทำไม error ที่ไม่จัดการทำให้ server ตาย try กับ catch ทำงานอย่างไร custom error และ Express middleware ตัวเดียวที่ตอบ status code ที่ถูกต้อง

What is an Exception | Error Types | Error Propagation | try and catch | Custom Errors | Error-handling Middleware | Status Codes | Assignment | อภิธานศัพท์

01

What is an Exception

exception หรือ error คือเหตุการณ์ที่ขัดขวางลำดับการทำงานปกติ สมมุติโปรแกรมควรรันบรรทัด 1 ถึง 50 แต่ข้อมูลทำให้บรรทัด 25 พัง ที่เหลือจึงรันไม่ได้

- error ถูกโยนออกมาเป็น object ที่เก็บรายละเอียด โยนได้เฉพาะ object ของ Error
- ถ้าไม่มีใครดัก **Node** จะหยุดโปรแกรมและจบด้วย code 1 **back end** ที่หยุดคือแอปที่ล่ม **front end** เรียกไม่ได้อีก
- error เกิดได้ในทุก layer แม้ SQL จะถูก เช่น database บอกว่าคอลัมน์ต้องไม่ซ้ำ แต่เรา insert ค่าซ้ำ

ทำไมต้องจัดการ:

- แอปไม่ล่ม จึงใช้งานต่อได้
- ผู้ใช้ได้ข้อความที่เป็นมิตร เช่น "ค่านี้ไม่ถูกต้อง"
- ข้อมูลสำคัญไม่หลุด error ที่ไม่ได้จัดการจะส่ง **stack trace** ที่อาจเผยชื่อ table หรือมากกว่านั้น
- **debug** และติดตามง่ายขึ้น

02

Error Types

Programming error

พิมพ์ผิดหรือ syntax ผิด พบน้อยแล้วเพราะ IDE ขีดเส้นใต้ก่อนรัน

Runtime error

code ที่ถูกต้องแต่พังตอนรัน x เป็นตัวเลข อีกห้าบรรทัดเป็นข้อความ แล้วเอาไปคูณ JavaScript ยอมให้ทำได้ จึงต้องดัก

Logical error

code รันได้แต่ได้คำตอบผิด exception ช่วยไม่ได้ ต้อง debug ไล่ทีละบรรทัดและดูค่า

Operational error

code ไม่ผิดแต่สภาพแวดล้อมมีปัญหา เช่น ไม่มีไฟล์ ต่อ database ไม่ได้ บางอย่างคาดได้ บางอย่างคาดไม่ได้ ต้องดักทั้งสองแบบ

- programming และ **logical error** คือบั๊กที่ต้องแก้ ส่วน **runtime** และ **operational error** คือสิ่งที่ต้องดัก
- ถ้าโยนบั๊กให้ AI แก้ คุณก็ยังไม่รู้ว่าผิดอะไร จึงไม่ได้เรียนรู้

03

Error Propagation

layer คือฟังก์ชันหนึ่งเรียกอีกฟังก์ชัน main เรียก a, a เรียก b ถ้า b โยน error แล้วไม่มีใครดัก error จะถูกส่งขึ้นไปที่ a แล้ว main แล้ว runtime ซึ่งจะหยุด ดักที่ระดับไหนก็ได้

test-exception-handling.js

```
import fs from 'node:fs';
function main(value = 0) {
  console.log(a(value));
}
function a(value) {
  return b(value);
}
function b(value) {
  if (value > 0) return ++value;
  return fs.readFileSync('test.txt'); // the file does not exist: throws
}
```

ไม่ดักเลย | ใน b() | ใน a() | ใน main()

```
main()                                  error ผ่านขึ้นไป
  a()                                   error ผ่านขึ้นไป
    b()                                 error เกิดที่นี่
runtime : หยุดโปรแกรม
```

โปรแกรม

ตาย exit code 1 ไม่มีอะไรหลัง error ถูกรัน

สำหรับ back end

server หยุดรอรับ ทุก client ถูกตัดขาด

ดักที่ระดับไหนก็ได้ ดักที่เดียวระดับสูงแปลว่ามีที่เดียวให้แก้

- ในโปรเจกต์ chain เดียวกันคือ **router**, **service**, **repository** ดักในแต่ละตัวก็ได้ หรือดักครั้งเดียวด้านบนใน app.js การดักที่เดียวดูแลง่ายกว่า

> ⚠ **Stack trace**
>
> stack trace ของ error ที่ไม่ถูกดักบอกไฟล์ บรรทัด และใครเรียกใคร มีประโยชน์กับเรา แต่ห้ามส่งให้ **client**

04

try and catch

ใส่ code ที่เสี่ยงใน try ถ้าโยน error การทำงานจะกระโดดไป catch และตัวแปรที่ตั้งชื่อไว้รับ object ของ error ถ้าไม่มีอะไรโยน catch จะถูกข้าม

try and catch

```
try {
  main(1);
  console.log('After main success');
  main(); // throws, so the next line is skipped
  console.log('After main error');
} catch (e) {
  console.log('Message:', e.message);
  console.log('Code:', e.code);
}
console.log('Program ended normally');
```

- main(1) ทำงานได้ แล้ว main() โยน error "After main error" จึงไม่ถูกพิมพ์ catch ทำงาน แล้วโปรแกรมจบปกติด้วย code 0
- object Error มาตรฐานมี message (ข้อความอ่านง่าย), name (ชนิด) และ stack (trace) Node เพิ่ม code (เช่น ENOENT) และ errno
- status ไม่ใช่ส่วนของ JavaScript เราเพิ่มเอง เพราะเป็นเรื่องของ **HTTP**
- ใน **REST API** เมื่อดักได้แล้ว ต้องสร้างคำตอบเอง: **JSON** พร้อม **status code** ที่ถูกต้อง

05

Custom Errors

สร้าง error เองได้ด้วย new Error และ message ถ้าอยากให้ทุก error มีรูปแบบเดียวกัน เขียน class ของเราที่ extends Error แล้วเพิ่ม status code

AppError

```
class AppError extends Error {
  errors = { 400: 'BAD_REQUEST', 401: 'UNAUTHORIZED', 403: 'FORBIDDEN',
             404: 'NOT_FOUND', 409: 'CONFLICT', 500: 'INTERNAL_SERVER_ERROR' };
  constructor(message, statusCode) {
    super(message); // the message is stored by the Error parent
    this.statusCode = statusCode;
    this.error = this.errors[statusCode];
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}
throw new AppError('Country with id 999 was not found', 404);
```

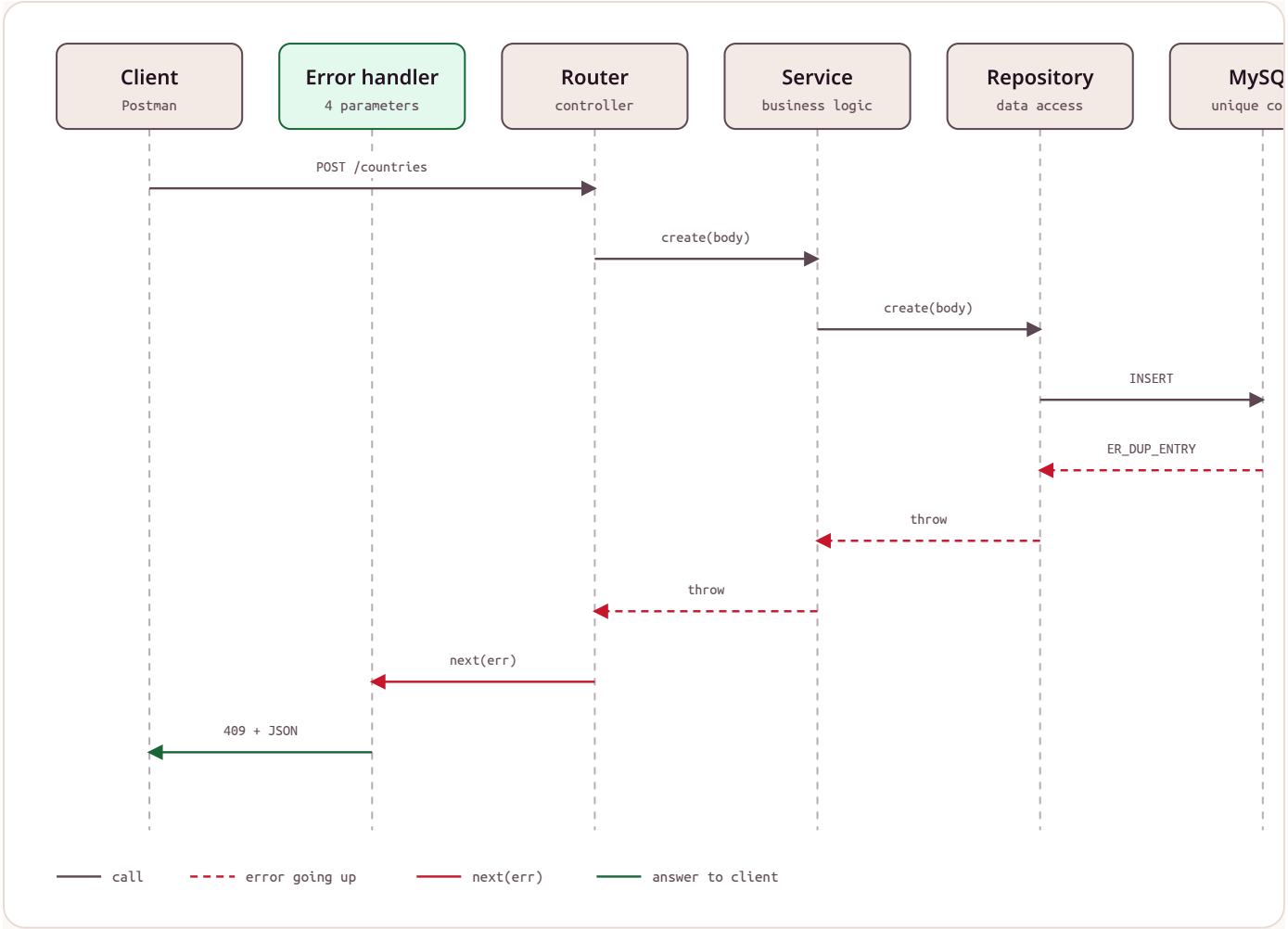
- super(message) ส่งข้อความให้ Error เก็บไว้ใน message เจ้าของ field นี้คือ class แม่
- ตาราง errors แปลง 404 เป็น NOT_FOUND เลือก 404 แล้วได้คำที่ถูกต้อง
- รูปแบบของ error เป็นการตัดสินใจร่วมกันระหว่าง front end กับ back end ตกลงเรื่อง field ตั้งแต่ต้น API ของ Node หลายตัวใช้ success flag กับ data

06

Error-handling Middleware

Express มีกลไกพิเศษ เขียน **error-handling middleware** ตัวเดียว แล้ว error จากทุก layer จะถูกส่งมาที่มันให้เอง ไม่ต้องเขียน try และ catch ทุกที่

- **middleware** ปกติมี (req, res, next) ส่วน error-handling มีสี่ตัว (err, req, res, next) Express เรียกเฉพาะเมื่อมี error ส่งมา



error-handler middleware

```
// error-handling middleware: four parameters
app.use((err, req, res, next) => {
  const status = err.code === 'ER_DUP_ENTRY' ? 409 : err.statusCode || err.status || 500;
  res.status(status).json({
    statusCode: status,
    message: err.message,
    path: req.originalUrl,
    timestamp: new Date().toISOString(),
  });
});
```

- อาจารย์เพิ่มอีกขั้น: duplicate entry error จาก database (code ER_DUP_ENTRY) ตอบด้วย 409 Conflict error อื่นที่มี status ใช้ตามนั้น ถ้าไม่มี status เลยให้เป็น 500
- คำตอบมี status code, ข้อความที่อ่านง่าย, **URL** ที่พัง (req.originalUrl) และ **timestamp**
- โปรเจกต์เล็กเขียนใน app.js ได้ เมื่อโตขึ้นให้แยกไปไฟล์ของมันแล้ว import เหมือน router

app.js

```
const app = express();
app.use(express.json());
app.use('/countries', countryRouter);
// ... all other routers
app.use(errorHandler); // last, after every router
```

ลำดับสำคัญ ลองสามแบบ

handler อยู่ท้ายสุด | ไม่มี handler | handler อยู่หน้าสุด

`app.use: express.json()` → `app.use: routers` → `app.use: error handler`

client ได้รับ

JSON พร้อม status code ที่ถูกต้อง

Body

```
{ "statusCode": 409, "message": "Dupli
```

error จาก **router** ใดก็ตามจะตกลงมาที่ handler

ลองให้เห็นเอง

ในโปรเจกต์สาธิตอาจารย์ตั้งคอลัมน์ country ให้ unique ใน database แล้ว POST Thailand สองครั้ง ถ้าไม่มี handler: ได้ 500 และ stack trace แบบ HTML ถ้ามี: ได้ JSON 409 ที่ **body** มี status, message, URL และเวลา

07

Status Codes

front end รู้ว่าเกิดอะไรขึ้นจาก status code จึงต้องเลือกให้ถูก ดูว่าใครทำผิด

400 Bad Request

client ส่งสิ่งที่ไม่ถูกต้อง เช่น ข้อความแทนตัวเลข หรือขาด field ใน body

401 Unauthorized

ยังไม่ login หรือไม่มี token

403 Forbidden

login แล้วแต่ไม่มีสิทธิ์ใช้ **endpoint** นี้ เช่น เฉพาะ admin

404 Not Found

ไม่มี **resource** นั้น เจอบ่อยที่สุด

409 Conflict

ข้อมูลขัดกับที่มีอยู่ เช่น ประเทศที่มีอยู่แล้ว

412 Precondition Failed

เงื่อนไขใน **header** ของ **request** ไม่ผ่าน เช่น ขาด header ที่จำเป็น

500 Internal Server Error

ฝั่งเราพัง ปกติไม่ควรตั้งใจส่ง

503 Service Unavailable

ตอนนี้บริการไม่พร้อมใช้งาน

- front end แปลง code เป็นข้อความให้ผู้ใช้ รหัสเหล่านี้ยังใช้ในโปรเจกต์รวมภายหลังด้วย

08

Assignment

ใบงานเป็นไฟล์ markdown ในเอกสารสัปดาห์ที่ 6 เปิดใน IDE โหมด preview แล้วอ่านให้ละเอียด ให้เริ่มโปรเจกต์ใหม่จาก template ของคุณและชี้ไปที่ database classicmodels

- ข้อ 1: จัดการ table offices เมื่อไม่เจอแถว ให้ตอบด้วยรูปแบบ error ตามที่ใบงานกำหนด

- ข้อ 2: สร้าง product line และจัดการ duplicate entry
- อย่างน้อยข้อ 1 ส่งในคลาส ส่วนที่เหลือมีกำหนดส่งทีหลัง ดูที่หน้าการบ้านของวิชา
- ทำตามใบงานแม้รูปแบบ error จะต่างจากตัวอย่างของอาจารย์ API ของ Node หลายตัวส่ง status พร้อม flag success หรือ error

> **ใช้ซ้ำ**
> ใบงานมี class error สำเร็จรูปให้ ใส่เข้า middleware ได้เลย ชื่อ field อาจต่าง เช่น error code กับ code ต้องทำให้ repository กับ middleware ตรงกัน

09

อภิธานศัพท์

API
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

back end
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

client
: ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

debug
: หาและแก้บั๊ก มักด้วยการไล่ code ทีละบรรทัดและดูค่า

endpoint
: method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

error-handling middleware
: middleware ที่มีสี่ parameter (err, req, res, next) Express เรียกเฉพาะเมื่อมี error ส่งมา

exception
: เหตุการณ์ที่ขัดขวางลำดับการทำงานปกติของโปรแกรม ถูกโยนออกมาเป็น object ที่เก็บรายละเอียด

Express
: web framework ขนาดเล็กของ Node.js เพิ่ม routing และ middleware บน http module ที่มีมาให้

front end
: ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

header

ข้อมูลเสริมของข้อความ ส่งเป็นบรรทัด name: value เช่น Content-Type

HTTP

HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

JSON

JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

logical error

code รันได้แต่ได้คำตอบผิด try และ catch ช่วยไม่ได้ ต้อง debug

middleware

ฟังก์ชัน (req, res, next) ที่ทำงานระหว่าง request กับ route handler แก้ request ตอบเอง หรือส่งต่อได้

Node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

operational error

code ไม่ผิด แต่สภาพแวดล้อมมีปัญหา เช่น ไม่มีไฟล์ หรือ database ล่ม

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

REST

แนวทางออกแบบ web API: resource ตั้งชื่อด้วย URL และเลือก action ด้วย HTTP method

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

runtime

สภาพแวดล้อมที่รัน code ของเรา Node.js เป็น runtime ของ JavaScript ไม่ใช่ภาษา

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

stack trace

รายการการเรียกฟังก์ชันพร้อมไฟล์และบรรทัดที่นำไปสู่ error บรรทัดบนสุดคือที่ที่ต้องดูก่อน

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

timestamp

ค่าวันที่และเวลา คอลัมน์อย่าง last_update ใส่เวลาปัจจุบันให้เองได้

URL

ที่อยู่ของ resource ประกอบด้วย protocol, host, port และ path เช่น http://localhost:3000/user